



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 19 de marzo del 2026

CALIFICACIÓN: _____

ACTIVIDAD REALIZADA

Optimización y carga de modelo de coche

Para la actividad de práctica se pidió seguir la misma lógica de lo realiza en el laboratorio, a partir de un modelo de un coche entregado en el previo en formato .fbx, se utilizó Blender



para separar el modelo en partes, siendo estas las cuatro llantas, el cofre y el resto del coche

Se guardaron estos modelos de forma individual obteniendo por resultado los siguientes archivos.

Nombre	Fecha de m...	Tipo	Tamaño
coche.mtl	17/03/2026 ...	Archivo MTL	3 KB
coche.obj	17/03/2026 ...	Object File	1,914 KB
cofre.mtl	18/03/2026 ...	Archivo MTL	1 KB
cofre.obj	18/03/2026 ...	Object File	13 KB
Llanta1.mtl	17/03/2026 ...	Archivo MTL	2 KB
Llanta1.obj	17/03/2026 ...	Object File	226 KB
Llanta2.mtl	17/03/2026 ...	Archivo MTL	2 KB
Llanta2.obj	17/03/2026 ...	Object File	226 KB
Llanta3.mtl	17/03/2026 ...	Archivo MTL	2 KB
Llanta3.obj	17/03/2026 ...	Object File	226 KB
Llanta4.mtl	17/03/2026 ...	Archivo MTL	2 KB
Llanta4.obj	17/03/2026 ...	Object File	226 KB

Pasando ahora a Visual Studio – OpenGL, los cambios realizados fueron los siguientes.

Window.h

Definimos las funciones getter publicas dentro de `class Window`, esto para que nos puedan

permitir que otros archivos usen las articulaciones.

```
// =====  
// GETTERS: Animaciones del Coche  
// Movimiento principal  
GLfloat getarticulacion1() { return articulacion1; } // Rota las 4 llantas al mismo tiempo  
GLfloat getmovCoche() { return movCoche; } // Traslación en el eje Z (Avance/Retroceso)  
// Carrocería  
GLfloat getarticulacion2() { return articulacion2; } // Abre y cierra el cofre  
// =====
```

Dentro de nuestra sección privada, declaramos las variables de tipo GLfloat, para almacenar los valores de grados.

```
// =====  
// VARIABLES: Control de ángulos de rotación (Goddard)  
GLfloat articulacion1, articulacion2, articulacion3, articulacion4, articulacion5;  
// =====
```

Window.cpp

Inicializamos las variables de articulación en la sección de `Window::Window(GLint windowHeight, GLint windowHeight)`

```
// =====  
// VARIABLES: Control de ángulos de rotación y traslación (Coche)  
GLfloat articulacion1, articulacion2, movCoche;  
// =====
```

Para nuestra lógica de articulaciones dentro de `void Window::ManejaTeclado(...)` manejamos las teclas de F y V para mover las llantas hacia adelante y hacia atrás respectivamente, mientras que las teclas G y B fueron asignadas para abrir y cerrar el cofre del coche teniendo un tope máximo de 60° y un mínimo de 0°

```
// =====  
// ANIMACIONES DEL COCHE: ROTACIÓN DE LAS LLANTAS  
// =====  
// Llantas (Giran las 4 al mismo tiempo) y Coche avanza - Teclas F (Adelante) y V (Atrás)  
if (key == GLFW_KEY_F) {  
    theWindow->articulacion1 += 5.0f; // Velocidad de giro de llantas  
    theWindow->movCoche -= 0.1f; // Velocidad de avance del coche (hacia Z negativa)  
}  
if (key == GLFW_KEY_V) {  
    theWindow->articulacion1 -= 5.0f; // Reversa de llantas  
    theWindow->movCoche += 0.1f; // Velocidad de retroceso del coche (hacia Z positiva)  
}  
// Cofre - Teclas G (Abrir) y B (Cerrar)  
if (key == GLFW_KEY_G) {  
    theWindow->articulacion2 += 2.0f; // Velocidad al abrir  
    if (theWindow->articulacion2 > 60.0f) theWindow->articulacion2 = 60.0f; // Tope Máximo  
}  
if (key == GLFW_KEY_B) {  
    theWindow->articulacion2 -= 2.0f; // Velocidad al cerrar  
    if (theWindow->articulacion2 < 0.0f) theWindow->articulacion2 = 0.0f; // Tope mínimo  
}  
// =====
```

Main (P05-317314975)

Declaramos las instancias globales de la clase Model, cada una representando una parte independiente de nuestro coche.

```
// =====  
Camera camera;  
// Modelos del Coche  
Model Carroceria;  
Model Cofre;  
Model Llanta1;  
Model Llanta2;  
Model Llanta3;  
Model Llanta4;  
// =====
```

Cargamos los modelos con el método LoadModel para cada una de nuestras instancias, este método es gracias a nuestra librería Assimp que lee archivos .obj, justo como los modelos que exportamos de blender.

```
// =====  
// CARGA DE MODELOS DEL COCHE  
Carroceria = Model();  
Carroceria.LoadModel("Models/coche.obj");  
Cofre = Model();  
Cofre.LoadModel("Models/cofre.obj");  
Llanta1 = Model();  
Llanta1.LoadModel("Models/Llanta1.obj");  
Llanta2 = Model();  
Llanta2.LoadModel("Models/Llanta2.obj");  
Llanta3 = Model();  
Llanta3.LoadModel("Models/Llanta3.obj");  
Llanta4 = Model();  
Llanta4.LoadModel("Models/Llanta4.obj");  
// =====
```

Optimizamos los colores, definiendo vectores antes de entrar al ciclo while principal para que no se creen en cada ciclo de reloj.

```
// =====  
// OPTIMIZACIÓN: Instanciamos los colores UNA SOLA VEZ  
glm::vec3 colorPiso = glm::vec3(0.310f, 0.314f, 0.329f);  
glm::vec3 colorChasis = glm::vec3(0.471f, 0.365f, 0.353f);  
glm::vec3 colorLlantas = glm::vec3(0.169f, 0.145f, 0.141f);  
glm::vec3 colorCofre = glm::vec3(0.471f, 0.310f, 0.290f);  
// =====
```

Instanciamos nuestro modelo completo, partimos del coche principal como el nodo padre, aquí es donde aplicamos el movimiento completo del coche mediante getmovCoche, para después instanciar nuestras 4 llantas como nodos hijos, siendo que el único cambio entre ellas que se da esta en la traslación de cada modelo para colocarse en los extremos del coche, finalmente instanciamos el nodo hijo final que será el cofre y asignamos la rotación en X para abrir y cerrarlo con lo ya definido en window.cpp

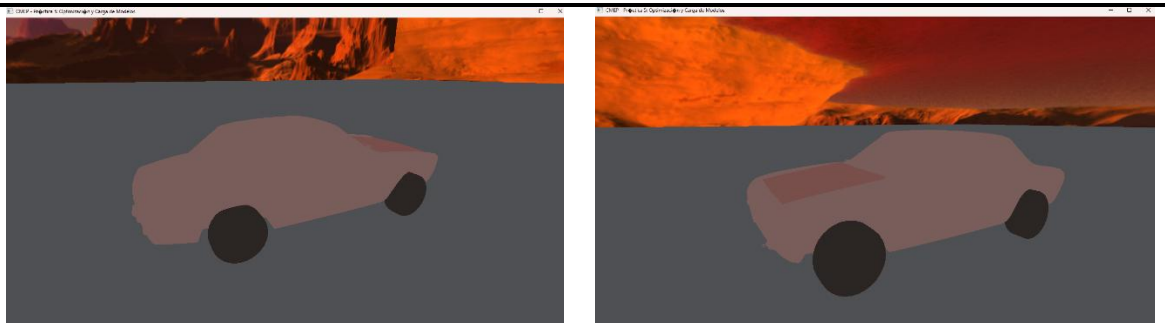
```
// =====  
// JERARQUÍA DEL COCHE
```

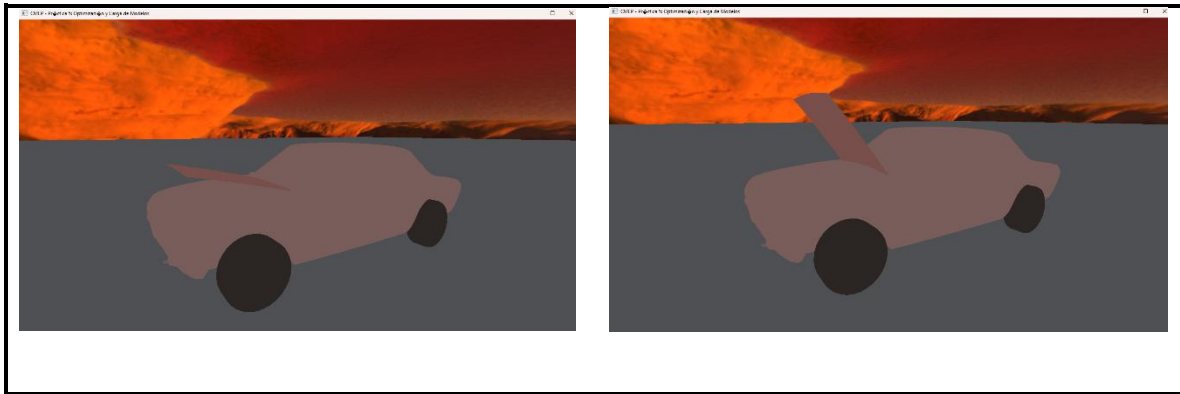
```
// =====
// -----
// 1. NODO PADRE: Chasis / Carrocería
model = glm::mat4(1.0);
// Traslación base. La variable getmovCoche() se suma a Z para hacer avanzar o retroceder todo el modelo.
model = glm::translate(model, glm::vec3(0.0f, -1.0f, -3.0f + mainWindow.getmovCoche()));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
modelaux = model; // GUARDAMOS EL PIVOTE CENTRAL para que lo hereden las llantas y el cofre
glUniform3fv(uniformColor, 1, glm::value_ptr(colorChasis));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carroceria.RenderModel();
// -----

// -----
// NODOS HIJOS (LLANTAS)
glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlantas)); // El color aplica para las 4 llantas
// NODO HIJO 1: Llanta Trasera Izquierda (Z Positiva)
model = modelaux;
model = glm::translate(model, glm::vec3(-3.0f, 0.3f, 4.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta1.RenderModel();
// NODO HIJO 2: Llanta Trasera Derecha (Z Positiva)
model = modelaux;
model = glm::translate(model, glm::vec3(3.0f, 0.3f, 4.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta2.RenderModel();
// NODO HIJO 3: Llanta Delantera Izquierda (Z Negativa)
model = modelaux;
model = glm::translate(model, glm::vec3(-3.0f, 0.3f, -6.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta3.RenderModel();
// NODO HIJO 4: Llanta Delantera Derecha (Z Negativa)
model = modelaux;
model = glm::translate(model, glm::vec3(3.0f, 0.3f, -6.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta4.RenderModel();
// NODO HIJO 5: Cofre
model = modelaux;
// Se traslada el cofre para que su pivote de rotación coincida con las bisagras
model = glm::translate(model, glm::vec3(0.0f, 2.7f, -4.2f));
// Rotación para abrir y cerrar el cofre (controlado con getarticulacion2)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorCofre));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cofre.RenderModel();
// -----
// =====
```

EJECUCIÓN

Teclas de movimiento: F,V – G,B





Para una mejor visualización se adjunta un link al GIF de drive.

https://drive.google.com/file/d/1vmJ_4giywcox-WO6W6FfG-St_ma3I2Ei/view?usp=sharing

PROBLEMAS PRESENTADOS

Para esta práctica no se presentó algún problema, contrariamente fue más rápida y sencilla de realizar que el ejercicio previo. La lógica de programación y la implementación de la jerarquía de nodos resultaron ser casi iguales en comparativa al ejercicio previo.

CONCLUSIONES

Sobre el ejercicio de clase

Con la realización de esta práctica pudimos reafirmar los conceptos de transformaciones jerárquicas hechos desde la práctica anterior. Al aplicar la lógica ya conocida a un modelo de coche, quedó evidenciado cómo un nodo padre (el chasis) puede gobernar la traslación de todo el conjunto de manera global, mientras que los nodos hijos (llantas y cofre) mantienen su independencia para rotar sobre sus propios ejes de forma local. Adicionalmente, un punto a destacar de esta sesión fue la optimización del código fuente; instanciar los vectores de color una sola vez fuera del ciclo de renderizado principal demostró nuevamente ser una buena práctica para hacer que el programa sea más eficiente en el consumo de memoria.

Comentarios generales

A diferencia de los primeros acercamientos con modelos externos, el flujo de trabajo en esta ocasión fue notablemente más rápido. Esto confirma que una gran parte de la complejidad en la computación gráfica sucede en la correcta preparación de los recursos: al tener las mallas bien separadas y los pivotes correctamente ajustados al origen desde el software de modelado (Blender), la implementación matemática en OpenGL utilizando la librería Assimp resulta ser muy directa y hasta el momento, sencilla.

Cierre.

En conclusión, los objetivos de optimización y carga de modelos complejos se cumplieron de manera exitosa. La práctica nos deja con una comprensión más de la jerarquía de nodos la cual sin duda será de gran utilidad y sentará las bases para la manipulación de escenarios tridimensionales más complejos en el desarrollo del proyecto final.

REFERENCIAS

- Ddiaz Design. (s.f.). *1965 Alfa Romeo Giulia Sprint GTA 1600* [Modelo 3D]. Sketchfab. Recuperado el 11 de marzo de 2026, de <https://sketchfab.com/3d-models/1965-alfa-romeo-giulia-sprint-gta-1600-460d44b163e24836bbefa48be9357ee1>